Dealer Flow Database Reference

End-to-end map of every table and column touched by the dealer lifecycle in the iTarang Next.js app — from a dealer filling out the onboarding wizard, through admin verification + correction + approval, into the dealer's own dashboard, and through the four-step lead creation flow.

Use this as a quick lookup when: - Debugging "what gets written when X happens" - Onboarding a new developer to the data model - Designing a new feature that touches an existing flow - Auditing the read/write path for a specific user action

Where the schema lives

Source	Path
Authoritative Drizzle schema	`src/lib/db/schema.ts`
Drizzle config	`drizzle.config.ts`
Drizzle client (postgres-js)	`src/lib/db/index.ts`
Database engine	PostgreSQL on AWS RDS (`DATABASE_URL`)
Storage (files)	Supabase Storage, bucket `dealer-documents`
Auth	Supabase Auth (session only); user profile + role live in AWS RDS `users` table

Drizzle property names in code map 1:1 to PostgreSQL column names via the explicit `pgTable` definitions — e.g. `companyName: text("company_name")`. Tables in this doc use the **PostgreSQL identifier** (snake_case) so they match `\d <table>` output and SQL queries you'd write against database-1.

Phase 1 — Dealer Onboarding Wizard

Public flow at `/dealer-onboarding` (no login required). State persists per draft via autosave; final submit promotes the row to `submitted`.

Pages and routes

Step	Component	What dealer enters
1. Company	`src/components/onboarding/steps/StepCompany.tsx`	Company name, type, GST, PAN, address; uploads GST cert + company PAN
2. Documents	`src/components/onboarding/steps/StepDocuments.tsx`	ITR (3y), bank statement (3m), 4 cheques, passport photo, Udyam cert
3. Ownership	`src/components/onboarding/steps/StepOwnership.tsx`	Owner / partner / director details; owner photo
4. Finance	`src/components/onboarding/steps/StepFinance.tsx`	Finance enable yes/no; sales manager details
5. Agreement	`src/components/onboarding/steps/StepAgreement.tsx`	Signing dates, dealer signatory, iTarang signatories
6. Review	`src/components/onboarding/steps/StepReview.tsx`	Final submit

API endpoints

Endpoint	File	Purpose
POST `/api/dealer-onboarding/save`	`src/app/api/dealer-onboarding/save/route.ts`	Autosave between steps
POST `/api/uploads/dealer-documents`	`src/app/api/uploads/dealer-documents/route.ts`	File upload to Supabase Storage

POST `/api/dealer/onboarding/submit`	`src/app/api/dealer/onboarding/submit/route.ts`	Final submit — promotes to `submitted`, writes all docs

Tables and columns written

`dealer_onboarding_applications`

Primary record for the dealer's application. One row per dealer attempt.

Column	Type	Phase written	Purpose
`id`	uuid PK	autosave	Application id; surfaces as `dealerId` everywhere downstream
`dealer_user_id`	uuid	autosave	Optional link to `users.id` if pre-provisioned
`company_name`	text	step 1	Legal entity name
`company_type`	text	step 1	`sole_proprietorship` / `partnership_firm` / `private_limited_firm`
`gst_number`	text	step 1	GSTIN
`pan_number`	text	step 1	Company PAN
`cin_number`	text	step 1	CIN (private limited only)
`business_address`	jsonb	step 1	`{ address, city, state, pincode }`
`registered_address`	jsonb	step 1	Same shape, optional
`finance_enabled`	bool	step 4	Drives Digio agreement requirement + Loan flow access
`owner_name` / `owner_phone` / `owner_landline` / `owner_email`	text	step 3	Primary contact; copied from first partner/director if not sole prop
`bank_name` / `account_number` / `beneficiary_name` / `ifsc_code`	text	step 3	Payout bank account
`sales_manager_name` / `sales_manager_email` / `sales_manager_mobile`	text/varchar	step 5	Mirrored under `provider_raw_response.agreement.salesManager`
`itarang_signatory_1_name` / `_email` / `_mobile`	text/varchar	step 5	iTarang signer for Digio
`itarang_signatory_2_name` / `_email` / `_mobile`	text/varchar	step 5	Optional second signer
`agreement_language`	varchar(30)	step 5	`english` / `hindi` / `bengali`
`agreement_status`	varchar(50)	step 5 + Digio webhook	`not_generated` / `sent_for_signature` / `completed` / `failed`
`provider_document_id` / `request_id` / `provider_signing_url` / `provider_raw_response`	text / jsonb	Digio integration	Digio identifiers + last raw payload
`stamp_status` / `stamp_certificate_ids`	varchar / jsonb	Digio integration	E-stamp tracking
`signed_at` / `signed_agreement_url` / `signed_agreement_storage_path`	timestamp / text	Digio webhook	Final signed PDF
`audit_trail_url` / `audit_trail_storage_path`	text	Digio webhook	Audit trail PDF
`submitted_at`	timestamp	submit	Lock-in moment for "submitted" state
`onboarding_status`	varchar(30)	autosave + submit + admin actions	See enum table below

`review_status`	varchar(30)	admin actions	See enum table below
`dealer_account_status`	varchar(30)	approve	`inactive` until approve, then `active`
`completion_status`	varchar(30)	admin actions	`pending` / `completed`
`correction_remarks`	text	admin Request Correction	Latest reviewer remark
`rejection_remarks`	text	admin Reject	Latest reject remark
`dealer_code`	text	approve	`ACC-YYYYMMDD-XXXXXX` (or shared parent's code if branch)
`is_branch_dealer`	bool	approve	True if approved against an existing GSTIN+PAN account
`approved_at` / `rejected_at`	timestamp	approve / reject	Audit timestamps
`created_at` / `updated_at`	timestamp	autosave	Standard timestamps

`dealer_onboarding_documents`

One row per uploaded file. Reused for original docs, agreement copies, and correction-round re-uploads.

Column	Type	Notes
`id`	uuid PK	
`application_id`	uuid FK → `dealer_onboarding_applications.id` ON DELETE CASCADE	
`document_type`	varchar(100)	`gst_certificate`, `company_pan`, `itr_3_years`, `bank_statement_3_months`, `undated_cheques`, `passport_photo`, `udyam_certificate`, `owner_photo`, `partnership_deed`, `mou_document`, `aoa_document`, `partner_photo_<n>`, `director_photo_<n>`
`bucket_name`	text	Always `dealer-documents`
`storage_path`	text	`<folder>/<timestamp>-<uuid>-<filename>`
`file_name` / `file_url` / `mime_type` / `file_size`	text / text / varchar / bigint	File metadata
`uploaded_by`	uuid	`users.id` if known
`doc_status`	varchar(30)	`uploaded` / `verified` / `rejected` / `pending_correction` (set by correction submit) / `superseded` (set by apply-correction)
`verification_status`	varchar(30)	`pending` / `verified` / `rejected`
`verified_at` / `verified_by`	timestamp / uuid	
`rejection_reason`	text	
`extracted_data` / `api_verification_results` / `metadata`	jsonb	OCR output, third-party verification responses, source tags
`uploaded_at` / `created_at` / `updated_at`	timestamp	

`dealer_agreement_signers`

Per-signer state for the Digio agreement.

Column	Type	Notes
`id`	uuid PK	
`application_id`	uuid FK CASCADE	
`signer_role`	varchar(50)	`dealer` / `itarang_1` / `itarang_2` / `financier`
`signer_name` / `signer_email` / `signer_mobile`	text	

`signing_method`	varchar(50)	`aadhaar_esign` / `electronic_signature`
`provider_document_id` / `request_id` / `provider_signer_identifier` / `provider_signing_url`	text	Digio identifiers
`signer_status`	varchar(50)	`pending` / `requested` / `signed` / `failed`
`signed_at` / `last_event_at`	timestamp	
`provider_raw_response`	jsonb	Last raw signer payload from Digio
`created_at` / `updated_at`	timestamp	

Indexes: `application_id`, `provider_document_id`, `signer_status`.

`dealer_agreement_events`

Append-only event log for the Digio agreement (one row per webhook event).

Column	Notes
`id` (uuid PK), `application_id` (FK CASCADE), `provider_document_id`, `request_id`	
`event_type` (varchar 100)	`agreement_requested` / `signed` / `expired` / etc.
`signer_role` (varchar 50)	Signer this event relates to (if any)
`event_status` (varchar 50)	
`event_payload` (jsonb)	Full raw webhook body
`created_at` (timestamp)	

Indexes: `application_id`, `provider_document_id`, `created_at`.

Phase 2 — Admin Dealer Verification

Admin role: `sales_head`. All routes guarded by `requireSalesHead()` in `src/lib/auth/requireSalesHead.ts`.

Pages

Page	Path	Route URL
Queue	`src/app/(dashboard)/admin/dealer-verification/page.tsx`	`/admin/dealer-verification`
Detail	`src/app/(dashboard)/admin/dealer-verification/[dealerId]/page.tsx`	`/admin/dealer-verification/[dealerId]`

API endpoints

Endpoint	File	Purpose
GET `/api/admin/dealer-verifications`	`…/route.ts`	Queue list (filtered by status, GST, etc.)
GET `/api/admin/dealer-verifications/[dealerId]`	`[dealerId]/route.ts`	Full detail (also returns latest correction round)
PATCH `/api/admin/dealer-verifications/[dealerId]`	same	Edit company / owner / bank / sales-manager fields
GET `/api/admin/dealer-verifications/[dealerId]/agreement-tracking`		Per-signer Digio status
POST `/api/admin/dealer-verifications/[dealerId]/initiate-agreement`		Build PDF + push to Digio
POST `/api/admin/dealer-verifications/[dealerId]/refresh-agreement`		Re-poll Digio + cache signed PDF + audit trail

POST `/api/admin/dealer-verifications/[dealerId]/duplicate-check`		GSTIN/PAN duplicate detection
POST `/api/admin/dealer-verifications/[dealerId]/audit-trail`		Download audit-trail PDF

Tables read: same set as Phase 1, plus `accounts` (for the duplicate / branch detection — looks up rows by `gstin` + `pan`).

PATCH writes mostly to `dealer_onboarding_applications` columns directly. Snapshot fields like `bank_branch`, `account_type`, owner residential address fields, and `sales_manager.*` are written into `provider_raw_response.submissionSnapshot.ownership` and `provider_raw_response.agreement.salesManager` respectively (jsonb merge).

Phase 3 — Request Correction (admin → dealer)

Admin clicks Request Correction in the modal, picks docs and fields and remarks, the system mints a tokenized magic link and emails the dealer.

Files

Layer	File
Modal	`src/components/admin/dealer-verification/RequestCorrectionDialog.tsx`
API	`src/app/api/admin/dealer-verifications/[dealerId]/request-correction/route.ts`
Email	`src/lib/email/sendDealerCorrectionNotificationEmail.ts`
Token helpers	`src/lib/onboarding/correction-token.ts`
Catalog	`src/lib/onboarding/correction-catalog.ts`

Tables written

`dealer_correction_rounds`

One row per round. Older open rounds are flipped to `superseded` when a new round is opened.

Column	Type	Notes
`id`	uuid PK	
`application_id`	uuid FK CASCADE	
`round_number`	integer	1, 2, 3 … per application
`status`	varchar(30)	`pending` → `submitted` → `applied` / `superseded`
`requested_by`	uuid	`users.id` of the admin who clicked the button
`remarks`	text NOT NULL	Free-text reviewer note
`requested_fields`	jsonb (string[])	Catalog field keys
`requested_documents`	jsonb (string[])	Catalog document keys
`dealer_submitted_at`	timestamp	Filled when dealer submits
`dealer_note`	text	Optional free text from dealer
`applied_by` / `applied_at`	uuid / timestamp	Filled by Update Application
`token_hash`	text UNIQUE	sha256(rawToken); raw token only ever in transit
`token_expires_at`	timestamp	now + 14 days
`created_at` / `updated_at`	timestamp	

Indexes: `application_id`, `status`, `token_hash`.

`dealer_correction_items`

One row per (round × item).

Column	Type	Notes
`id`	uuid PK	
`round_id`	uuid FK CASCADE	
`kind`	varchar(20)	`field` or `document`
`key`	varchar(100)	Field column name (`companyName`) or doc type (`gst_certificate`)
`previous_value`	text	Pre-correction column value (fields only)
`new_value`	text	Dealer's new value (fields only)
`previous_document_id`	uuid	FK to existing `dealer_onboarding_documents.id` (docs only)
`new_document_id`	uuid	FK to dealer's new `dealer_onboarding_documents.id` (docs only)
`created_at`	timestamp	

Indexes: `round_id`.

Side effects on `dealer_onboarding_applications`

```
onboarding_status      = 'correction_requested'
review_status          = 'under_correction'
dealer_account_status  = 'inactive'
completion_status      = 'pending'
correction_remarks     = <admin's remark>
updated_at             = now()
```

Phase 4 — Dealer Responds to Correction

Public, token-authenticated. No Supabase login required.

Files

Layer	File
Page	`src/app/onboarding/correct/[token]/page.tsx`
API GET (load)	`src/app/api/onboarding/correct/[token]/route.ts`
API POST (submit)	same file
File upload reused	`src/app/api/uploads/dealer-documents/route.ts`
Component reused	`src/components/onboarding/FileUploadCard.tsx`

Tables written by POST

- New rows in `dealer_onboarding_documents` for each re-uploaded doc, with `doc_status = 'pending_correction'`, `metadata.correctionRoundId = <round>`.
- `dealer_correction_items.new_document_id` linked to the new doc rows.
- `dealer_correction_items.new_value` set for each updated field.
- `dealer_correction_rounds.status = 'submitted'`, `dealer_submitted_at = now()`, `dealer_note` (optional).

The original `dealer_onboarding_applications` columns are **not** touched yet — only when admin applies the round (Phase 5).

Phase 5 — Apply Correction (admin)

Admin reviews the diff in the new Correction Response panel and clicks **Update Application**.

Files

Layer	File
Panel	`src/components/admin/dealer-verification/CorrectionResponsePanel.tsx`
API	`src/app/api/admin/dealer-verifications/[dealerId]/apply-correction/route.ts`

What happens

1. For each item with `kind = 'field'`, write `new_value` onto the matching column of `dealer_onboarding_applications`.
2. For each item with `kind = 'document'`: - Mark `previous_document_id` → `doc_status = 'superseded'`. - Mark `new_document_id` → `doc_status = 'uploaded'`, `verification_status = 'pending'`.
3. Update `dealer_correction_rounds`: `status = 'applied'`, `applied_at = now()`, `applied_by = admin.id`.
4. Update `dealer_onboarding_applications`: - `onboarding_status = 'submitted'` ← unlocks Approve - `review_status = 'agreement_completed'` if finance-enabled and `agreement_status = 'completed'`, else `'under_review'` - `completion_status = 'completed'` if agreement completed, else `'pending'` - `correction_remarks = NULL`

Phase 6 — Approve / Reject

Approve

File	`src/app/api/admin/dealer-verifications/[dealerId]/approve/route.ts`

Gates (reject if any fails):

1. `onboarding_status === 'submitted'`.
2. Not already `approved`.
3. If `finance_enabled`: `agreement_status === 'completed'` AND `review_status === 'agreement_completed'`.
4. If `finance_enabled`: signed-agreement PDF + audit-trail PDF cached (lazy-fetched via `ensureDealerSignedAgreementUrl` / `ensureDealerAuditTrailUrl`).
5. GSTIN/PAN classification: `duplicate` / `pan-mismatch` → 409 hard block; `branch` → reuse existing account; `none` → fresh account.

On success, writes:

Table	Action
`accounts`	INSERT (or reuse existing for branch dealers) — id = `dealer_code`
`users`	INSERT auth user with role `dealer` linked to `dealer_id`
`dealer_onboarding_applications`	`onboarding_status='approved'`, `review_status='approved'`, `dealer_account_status='active'`, `completion_status='completed'`, `approved_at=now()`, `dealer_code=<code>`, `is_branch_dealer` flag, `correction_remarks=NULL`, `rejection_remarks=NULL`

Side effects: Supabase auth user provisioned; welcome email sent via `src/lib/email/sendDealerWelcomeEmail.ts` with the temporary password, signed-agreement attachment, and audit-trail attachment.

Reject

File	`src/app/api/admin/dealer-verifications/[dealerId]/reject/route.ts`

Writes:

```
onboarding_status      = 'rejected'
review_status          = 'rejected'
dealer_account_status  = 'inactive'
rejection_remarks      = <admin remark>
rejected_at            = now()
```

No accounts row is created. Email goes out via `sendDealerRejectionNotificationEmail`.

Phase 7 — Dealer Logs In → Dashboard

Auth via Supabase. Middleware (`src/middleware.ts`) reads role from `app_metadata.role`, falls back to `users.role` lookup.

Pages

URL	File
`/dealer-portal`	`src/app/(dashboard)/dealer-portal/page.tsx`

Component: `src/components/dealer-dashboard/DealerDashboard.tsx`.

API endpoints called on dashboard mount

Endpoint	File	Tables read
GET `/api/dealer/stats`	`src/app/api/dealer/stats/route.ts`	`users`, `leads`, `inventory`, `loan_applications`, `dealer_onboarding_applications`
GET `/api/dealer/coupons/summary`	`src/app/api/dealer/coupons/summary/route.ts`	`coupon_codes`
GET `/api/dealer/leads`	`src/app/api/dealer/leads/route.ts`	`leads`
GET `/api/dealer/loans`	`src/app/api/dealer/loans/route.ts`	`loan_files`
GET `/api/dealer/assets`	`src/app/api/dealer/assets/route.ts`	`deployed_assets`
GET `/api/dealer/loan-facilitation/stats`	`…/loan-facilitation/stats/route.ts`	`loan_applications`, `leads`
GET `/api/dealer/onboarding/status`	`…/onboarding/status/route.ts`	`dealer_onboarding_applications`

Tables driving dashboard cards

Card	Table	Aggregation
Stock Available	`inventory`	COUNT(*)
Stock Deployed	`deployed_assets`	COUNT(*) WHERE `status='deployed'`
Total Customers	`leads`	COUNT(*) WHERE `dealer_id=?`
Conversion Rate	`leads`	converted / total × 100
Loan Applied	`loan_applications`	COUNT(*) joined leads on `dealer_id`
On-time Payment	`loan_files`	SUM(`total_paid`)
Coupons by status	`coupon_codes`	COUNT(*) GROUP BY status
Recent Leads (5)	`leads`	ORDER BY `created_at` DESC LIMIT 5

Logout

Endpoint	File	DB writes
GET/POST `/api/auth/logout`	`src/app/api/auth/logout/route.ts`	None — only clears `sb-*` cookies

Sub-pages under `/dealer-portal/*`

URL	Purpose
`/dealer-portal`	Dashboard cards + recent leads
`/dealer-portal/leads`	Lead list / filter
`/dealer-portal/leads/drafts`	Draft leads
`/dealer-portal/leads/new`	Create lead — Step 1
`/dealer-portal/leads/[id]/kyc`	Step 2 — KYC
`/dealer-portal/leads/[id]/borrower-consent`	Step 3 — Consent
`/dealer-portal/leads/[id]/product-selection`	Step 4 — Product + coupon
`/dealer-portal/leads/[id]/options`	Finance/lease options
`/dealer-portal/leads/[id]/step-5`	Step 5 — OTP + dispatch
`/dealer-portal/loans`	Loan processing (finance-enabled only)
`/dealer-portal/loans/facilitation`	Facilitation queue
`/dealer-portal/loans/management`	Active/closed loans
`/dealer-portal/assets` / `/batteries`	Deployed assets
`/dealer-portal/orders`	OEM purchase orders
`/dealer-portal/inventory`	Stock
`/dealer-portal/service`	Service tickets
`/dealer-portal/campaigns/new`	SMS / WhatsApp campaigns
`/dealer-portal/onboarding-status`	Read-only onboarding progress

Phase 8 — Lead Creation Step 1

URL	`/dealer-portal/leads/new`
Page	`src/app/(dashboard)/dealer-portal/leads/new/page.tsx`
API	POST `/api/leads/create` (modes: `initializeDraft` / `commitStep`)

Tables written

`leads`

The fat fact table for every customer the dealer touches. Columns most relevant to step 1:

Column	Notes
`id` (text PK)	`LEAD-XXXXXX`
`reference_id` (text)	`IT-YYYY-NNNNNNN` displayed to dealers
`dealer_id` (text FK → `accounts.id`)	Owning dealer
`uploader_id` (uuid FK → `users.id`)	Who created the lead
`lead_source` (text)	`dealer_referral` / etc.
`full_name` , `phone` , `mobile`	Normalized to +91 format
`owner_name` , `owner_contact`	Mirror of borrower for legacy flows

`current_address`, `permanent_address`, `is_current_same`	
`dob`, `father_or_husband_name`	Required ≥ 18 yrs
`product_category_id`, `product_type_id`, `primary_product_id`	Catalog linkage
`interest_level`	`hot` / `warm` / `cold`
`lead_score`	Derived: hot=90 / warm=60 / cold=30
`vehicle_rc`, `vehicle_ownership`, `vehicle_owner_name`, `vehicle_owner_phone`	Required for vehicle categories
`interested_in` (jsonb)	Array of additional interests
`payment_method`	`cash` / `upfront` / `finance` / `other_finance` / `dealer_finance`
`status`	`INCOMPLETE` (draft) → `ACTIVE` (committed) → `ABANDONED`
`lead_status`	`new` after creation
`kyc_status`	`not_started` (default) / `not_required` (cash/upfront)
`workflow_step`	`1` for finance, `3` for cash/upfront (skips KYC)
`created_at`, `updated_at`	

`personal_details`

Inserted on draft init, updated alongside leads.

Column	Notes
`id` (uuid PK)	
`lead_id` (FK → `leads.id`)	
`dob`, `father_husband_name`, `local_address`	Mirrors of `leads.*`

`audit_logs`

Append-only.

Column	Notes
`id` (text PK)	`AUDIT-<timestamp>`
`entity_type`	`lead`
`entity_id`	The lead id
`action`	`LEAD_CREATED_STEP1` etc.
`changes` (jsonb)	Full payload at the moment of write
`performed_by` (uuid)	`users.id`
`timestamp`	

Phase 9 — Lead Step 2 (KYC)

URL	**`/dealer-portal/leads/[id]/kyc`**
Page	`src/app/(dashboard)/dealer-portal/leads/[id]/kyc/page.tsx`
Access check API	GET `/api/kyc/[leadId]/access-check`
Doc upload API	POST `/api/dealer/upload` (formData with `documentType`, `docType`)

Tables touched

`kyc_documents`

One row per (lead, applicant, doc_type).

Column	Notes
`id` (text PK)	`KYCDOC-…`
`lead_id` (FK)	
`doc_for`	`customer` / `borrower` / `co_borrower`
`doc_type`	`aadhaar_front`, `aadhaar_back`, `pan_card`, `address_proof`, `rc_copy`, `bank_statement`, `cheque_1..4`, …
`file_url`, `file_name`, `file_size`, `mime_type`	
`verification_status`	`pending` / `in_progress` / `success` / `failed` / `awaiting_action`
`failed_reason`	
`ocr_data`, `api_response` (jsonb)	
`verified_at`, `uploaded_at`, `updated_at`	

`kyc_verifications`

One row per verification attempt.

Column	Notes
`id` (text PK)	`KYCVER-…`
`lead_id` (FK)	
`verification_type`	`aadhaar` / `pan` / `bank` / `address` / `rc` / `mobile` / `cibil` / `photo`
`applicant`	`primary` / `co_borrower`
`status`	`pending` → `initiating` → `awaiting_action` → `in_progress` → `success` / `failed`
`api_provider`	`decentro` / `surepass` / `vahan`
`api_request`, `api_response` (jsonb)	
`failed_reason`, `match_score`, `retry_count`	
`submitted_at`, `completed_at`	
`admin_action`, `admin_action_by`, `admin_action_at`, `admin_action_notes`	Admin override fields

`digilocker_transactions`

Decentro eAadhaar fetch flow.

Column	Notes
`id` (text PK)	`DIGI-…`
`lead_id`, `verification_id` (FKs)	
`reference_id`, `decentro_txn_id`, `session_id`	
`status`	`initiated` → `link_sent` → `link_opened` → `consent_given` → `document_fetched` / `failed` / `expired`
`customer_phone`, `customer_email`, `digilocker_url`, `short_url`, `notification_channel`	
`link_sent_at`, `link_opened_at`, `customer_authorized_at`, `expires_at`	
`digilocker_raw_response`, `aadhaar_extracted_data`, `cross_match_result` (jsonb)	
`aadhaar_pdf` (bytea)	Raw Aadhaar PDF from DigiLocker
`sms_message_id`, `sms_delivered_at`, `sms_failed_reason`, `sms_attempts`	

`consent_records`

Borrower consent / eSign lifecycle.

Column	Notes
`id` (text PK)	`CONSENT-…`
`lead_id` (FK), `consent_for`	`primary` / `co_borrower`
`consent_type`	`digital` / `manual` / `sms` / `whatsapp`
`consent_status`	See enum table below
`consent_token`, `consent_link_url`, `consent_link_sent_at`, `consent_delivery_channel`	
`esign_transaction_id`, `signed_consent_url`, `generated_pdf_url`, `signed_at`, `signer_aadhaar_masked`, `esign_retry_count`, `esign_error_message`	
`verified_by`, `verified_at`, `admin_viewed_by`, `admin_viewed_at`	

`kyc_data_audit`

Field-level provenance tracking.

Column	Notes
`id` (text PK)	`KYCAUD-…`
`lead_id`, `field_name`, `field_value`	
`data_source`	`ocr` / `api` / `manual`
`entered_by`, `entered_at`, `reason`	

`leads` (updates)

Step 2 updates: `kyc_status`, `kyc_score`, `kyc_completed_at`, `has_co_borrower`, `has_additional_docs_required`, `interim_step_status`, `kyc_draft_data`, `consent_status`, `updated_at`.

Phase 10 — Lead Step 3 (Borrower Consent)

URL	`/dealer-portal/leads/[id]/borrower-consent`
Page	`src/app/(dashboard)/dealer-portal/leads/[id]/borrower-consent/page.tsx`

Tables: same `consent_records` shape from Phase 9. Updates `leads.consent_status`.

Phase 11 — Lead Step 4 (Product Selection + Coupon)

URL	`/dealer-portal/leads/[id]/product-selection`
Page	`src/app/(dashboard)/dealer-portal/leads/[id]/product-selection/page.tsx`
Access API	GET `/api/lead/[id]/step-4-access`
Validate coupon	POST `/api/kyc/[leadId]/validate-coupon`
Release coupon	POST `/api/kyc/[leadId]/release-coupon`

Tables

`coupon_codes`

Column	Notes
`id` (text PK)	`COUPON-…`
`code` (text)	The redeemable code
`dealer_id` (FK → `accounts.id`)	
`status`	`available` / `reserved` / `used` / `expired` / `revoked`
`discount_type`	`flat` / `percentage`
`discount_value` , `max_discount_cap` , `min_amount`	numeric
`used_by_lead_id` , `used_by`	FKs once consumed
`validated_at` , `used_at` , `expires_at` , `created_at`	

`coupon_batches`

Column	Notes
`id` , `dealer_id` , `prefix` , `coupon_value` , `total_quantity` , `expiry_date` , `status` , `created_by` , `created_at` , `updated_at`	

`coupon_audit_log`

Column	Notes
`id` , `coupon_id` , `action` (`reserved` / `released` / `used` / `expired`), `old_status` , `new_status` , `lead_id` , `performed_by` , `ip_address` , `notes` , `created_at`	

`product_selections`

Column	Notes
`id` (text PK)	`PS-…`
`lead_id` (FK)	
`battery_serial` , `charger_serial` , `paraphernalia` (jsonb)	
`category` , `sub_category`	
`battery_price` , `charger_price` , `paraphernalia_cost` , `dealer_margin` , `final_price`	numeric
`payment_mode`	`cash` / `finance`
`admin_decision`	`pending` → `dealer_confirmed` → `sanctioned` / `rejected`
`submitted_by` , `submitted_at` , `created_at` , `updated_at`	

`leads` (updates)

`coupon_code` (text), `coupon_status` (`reserved` → `used`), `workflow_step = 4` , `updated_at` .

Phase 12 — Lead Step 5 (OTP + Dispatch)

URL	`/dealer-portal/leads/[id]/step-5`
Send OTP	POST `/api/lead/[id]/step-5/otp/send`
Confirm OTP	POST `/api/lead/[id]/step-5/otp/confirm`

Tables

`otp_confirmations`

Column	Notes
`id` (text PK)	`OTP-…`
`lead_id` (FK)	
`otp_type`	`dispatch_confirmation`
`otp_hash`	sha256 of OTP
`phone_sent_to`	
`created_at`, `expires_at`	
`send_count`, `attempt_count`, `locked_until`	Anti-bruteforce
`is_used`, `used_at`, `used_by`	

`loan_sanctions`

Column	Notes
`id` (text PK)	`LS-…`
`lead_id`, `product_selection_id` (FKs)	
`loan_amount`, `down_payment`, `emi`, `tenure_months`, `roi`	numeric
`loan_approved_by`	NBFC / bank name
`loan_file_number`	
`status`	`sanctioned` / `rejected` / `dealer_approved`
`sanctioned_by`, `sanctioned_at`, `dealer_approved`, `dealer_approved_at`	

`loan_applications`

Admin-driven facilitation post-KYC.

Column	Notes
`id` (text PK)	`LOAN-APP-…`
`lead_id` (FK)	
`applicant_name`, `loan_amount`, `documents_uploaded`	
`company_validation_status`, `facilitation_fee_status`, `application_status`	enum-like
`facilitation_fee_amount`, `created_by`, `created_at`, `updated_at`	

`facilitation_payments`

Column	Notes
`id`, `lead_id`, `payment_method`	
`facilitation_fee_base_amount`, `facilitation_fee_discount_amount`, `facilitation_fee_final_amount`	numeric
`payment_status` (`pending` / `paid`), `payment_date`, `transaction_id`, `created_at`, `updated_at`	

`loan_files`

Active loan record after sanction; drives the dealer's Loan Management page.

Column	Notes
`id`, `dealer_id`, `lead_id` (FKs)	
`loan_status`	`active` / `closed` / etc.
`loan_amount`, `disbursal_status`, `total_paid`, `overdue_days`	numeric

`after_sales_records`

Warranty + post-dispatch service tracking.

Column	Notes
`id`, `lead_id`, `dispatched_at`, `warranty_expires_at`, `service_status`	

`leads` (updates)

`workflow_step = 5`, `kyc_status` advances to `loan_sanctioned` → `sold`, `updated_at`.

Status enum reference

`dealer_onboarding_applications.onboarding_status`

`draft` → `submitted` → (`correction_requested` → back to `submitted`)* → `approved` / `rejected`

`dealer_onboarding_applications.review_status`

`pending` → `under_review` → `under_correction` / `agreement_completed` → `approved` / `rejected`

`dealer_onboarding_applications.dealer_account_status`

`inactive` (default) → `active` (only after approve)

`dealer_onboarding_applications.agreement_status`

`not_generated` → `sent_for_signature` → `completed` / `failed`

`dealer_correction_rounds.status`

`pending` → `submitted` → `applied` / `superseded`

`leads.status`

`INCOMPLETE` → `ACTIVE` / `ABANDONED`

`leads.kyc_status`

`not_started` / `not_required` → `pending_documents` → `documents_submitted` → `pending_verification` → `pending_itarang_reverification` → `kyc_approved` → `loan_sanctioned` → `sold` (or `loan_rejected` / `closed_loan_rejected`)

`leads.consent_status`

`awaiting_signature` → `link_sent` → `link_opened` → `digitally_signed` / `esign_completed` → `verified` (or `manual_uploaded` → `verified`, or `admin_rejected` / `expired`)

`leads.workflow_step`

1 (info) → 2 (KYC) → 3 (consent) → 4 (product + coupon) → 5 (OTP + dispatch)

`coupon_codes.status`

`available` → `reserved` (assigned to lead) → `used` (post-payment) / `expired` / `revoked`

Cross-cutting tables

Table	Used by	Why
`users`	Every authenticated request	Auth identity, role, dealer linkage
`accounts`	Approve, dealer dashboard, leads, coupons	Dealer legal entity
`audit_logs`	Many writes	Append-only audit trail
`dealer_onboarding_documents`	Onboarding + correction round + agreement copy	Single bucket for all dealer docs
`dealer_correction_rounds` / `dealer_correction_items`	Correction flow only	Per-round audit + dealer response storage

---

Converting this doc to PDF

Markdown was deliberate. Pick whichever path is easiest:

- **VS Code**: install the *Markdown PDF* extension (yzane.markdown-pdf) → right-click the file → Markdown PDF: Export (pdf).
- **Pandoc** (if installed): `pandoc docs/dealer-flow-database-reference.md -o dealer-flow.pdf --pdf-engine=xelatex`.
- **Browser**: open in any Markdown previewer (e.g. Obsidian, Typora) → Print → Save as PDF.
- **GitHub** (if pushed): the `.md` renders natively; browser Print → Save as PDF gives a clean version.

The Markdown is the source of truth — re-export the PDF whenever this file changes rather than maintaining a separate copy.